五、MySQL 数据库

(一) 建表规约

1. 【强制】表达是与否概念的字段，必须使用 is_xxx 的方式命名，数据类型是 unsigned tinyint（1 表示是，0 表示否）。
   注意：POJO 类中的任何布尔类型的变量，都不要加 is 前缀，所以，需要在<resultMap>设置从 is_xxx 到 Xxx 的映射关系。数据库表示是与否的值，使用 tinyint 类型，坚持 is_xxx 的命名方式是为了明确其取值含义与取值范围。
   说明：任何字段如果为非负数，必须是 unsigned。
   正例：表达逻辑删除的字段名 is_deleted，1 表示删除，0 表示未删除。

2. 【强制】表名、字段名必须使用小写字母或数字，禁止出现数字开头禁止两个下划线中间只出现数字。数据库字段名的修改代价很大，因为无法进行预发布，所以字段名称需要慎重考虑。
   说明：MySQL 在 Windows 下不区分大小写，但在 Linux 下默认是区分大小写。因此，数据库名、表名、字段名，都不允许出现任何大写字母，避免节外生枝。
   正例：aliyun_admin，rdc_config，level3_name
   反例：AliyunAdmin，rdcConfig，level_3_name

3. 【强制】表名不使用复数名词。
   说明：表名应该仅仅表示表里面的实体内容，不应该表示实体数量，对应于 DO 类名也是单数形式，符合表达习惯。

4. 【强制】禁用保留字，如 desc、range、match、delayed 等，请参考 MySQL 官方保留字。

5. 【强制】主键索引名为 pk_字段名；唯一索引名为 uk_字段名；普通索引名则为 idx_字段名。
   说明：pk_即 primary key；uk_即 unique key；idx_即 index 的简称。

6. 【强制】小数类型为 decimal，禁止使用 float 和 double。
   说明：在存储的时候，float 和 double 都存在精度损失的问题，很可能在比较值的时候，得到不正确的结果。如果存储的数据范围超过 decimal 的范围，建议将数据拆成整数和小数并分开存储。

7. 【强制】如果存储的字符串长度几乎相等，使用 char 定长字符串类型。

8. 【强制】varchar 是可变长字符串，不预先分配存储空间，长度不要超过 5000，如果存储长度大于此值，定义字段类型为 text，独立出来一张表，用主键来对应，避免影响其它字段索引率。

9. 【强制】表必备三字段：id，create_time，update_time。
   说明：其中 id 必为主键，类型为 bigint unsigned、单表时自增、步长为 1。create_time，update_time 的类型均为 datetime 类型，如果要记录时区信息，那么类型设置为 timestamp。

10. 【强制】在数据库中不能使用物理删除操作，要使用逻辑删除。
    说明：逻辑删除在数据删除后可以追溯到行为操作。不过会使得一些情况下的唯一主键变得不唯一，需要根据情况来酌情解决。

11. 【推荐】表的命名最好是遵循“业务名称_表的作用”。
    正例：alipay_task / force_project / trade_config / tes_question

12. 【推荐】库名与应用名称尽量一致。

13. 【推荐】如果修改字段含义或对字段表示的状态追加时，需要及时更新字段注释。

14. 【推荐】字段允许适当冗余，以提高查询性能，但必须考虑数据一致。冗余字段应遵循：
    1）不是频繁修改的字段。
    2）不是唯一索引的字段。
    3）不是 varchar 超长字段，更不能是 text 字段。
    正例：各业务线经常冗余存储商品名称，避免查询时需要调用 IC 服务获取。

15. 【推荐】单表行数超过 500 万行或者单表容量超过 2GB，才推荐进行分库分表。
说明：如果预计三年后的数据量根本达不到这个级别，请不要在创建表时就分库分表。

16. 【参考】合适的字符存储长度，不但节约数据库表空间、节约索引存储，更重要的是提升检索速度。
正例：无符号值可以避免误存负数，且扩大了表示范围：

对象	年龄区间	类型	字节	表示范围
人	150 岁之内	tinyint unsigned	1	无符号值：0 到 255
龟	数百岁	smallint unsigned	2	无符号值：0 到 65535
恐龙化石	数千万年	int unsigned	4	无符号值：0 到约 43 亿
太阳	约 50 亿年	bigint unsigned	8	无符号值：0 到约 10 的 19 次方

(二) 索引规约

1. 【强制】业务上具有唯一特性的字段，即使是组合字段，也必须建成唯一索引。
说明：不要以为唯一索引影响了 insert 速度，这个速度损耗可以忽略，但提高查找速度是明显的；另外，即使在应用层做了非常完善的校验控制，只要没有唯一索引，根据墨菲定律，必然有脏数据产生。

2. 【强制】超过三个表禁止 join。需要 join 的字段，数据类型保持绝对一致；多表关联查询时，保证被关联的字段需要有索引。
说明：即使双表 join 也要注意表索引、SQL 性能。

3. 【强制】在 varchar 字段上建立索引时，必须指定索引长度，没必要对全字段建立索引，根据实际文本区分度决定索引长度。
说明：索引的长度与区分度是一对矛盾体，一般对字符串类型数据，长度为 20 的索引，区分度会高达 90%以上，可以使用 count(distinct left(列名，索引长度)) / count(*) 的区分度来确定。

4. 【强制】页面搜索严禁左模糊或者全模糊，如果需要请走搜索引擎来解决。
说明：索引文件具有 B-Tree 的最左前缀匹配特性，如果左边的值未确定，那么无法使用此索引。

5. 【推荐】如果有 order by 的场景，请注意利用索引的有序性。order by 最后的字段是组合索引的一部分，并且放在索引组合顺序的最后，避免出现 filesort 的情况，影响查询性能。
正例：where a = ? and b = ? order by c；索引：a_b_c
反例：索引如果存在范围查询，那么索引有序性无法利用，如：WHERE a > 10 ORDER BY b；索引 a_b 无法排序。

6. 【推荐】利用覆盖索引来进行查询操作，避免回表。
说明：如果一本书需要知道第 11 章是什么标题，会翻开第 11 章对应的那一页吗？目录浏览一下就好，这个目录就是起到覆盖索引的作用。
正例：能够建立索引的种类分为主键索引、唯一索引、普通索引三种，而覆盖索引只是一种查询的一种效果，用 explain 的结果，extra 列会出现：using index。

7. 【推荐】利用延迟关联或者子查询优化超多分页场景。
说明：MySQL 并不是跳过 offset 行，而是取 offset+N 行，然后返回放弃前 offset 行，返回 N 行，那当 offset 特别大的时候，效率就非常的低下，要么控制返回的总页数，要么对超过特定阈值的页数进行 SQL 改写。
正例：先快速定位需要获取的 id 段，然后再关联：
SELECT t1.* FROM 表 1 as t1 , (select id from 表 1 where 条件 LIMIT 100000 , 20) as t2 where t1.id = t2.id

8. 【推荐】SQL 性能优化的目标：至少要达到 range 级别，要求是 ref 级别，如果可以是 const 最好。
说明：
1）consts 单表中最多只有一个匹配行（主键或者唯一索引），在优化阶段即可读取到数据。

2）ref 指的是使用普通的索引（normal index）。
3）range 对索引进行范围检索。

反例：explain 表的结果，type = index，索引物理文件全扫描，速度非常慢，这个 index 级别比较 range 还低，与全表扫描是小巫见大巫。

9. 【推荐】建组合索引的时候，区分度最高的在最左边。

正例：如果 where a = ? and b = ?，a 列的几乎接近于唯一值，那么只需要单建 idx_a 索引即可。

说明：存在非等号和等号混合判断条件时，在建索引时，请把等号条件的列前置。如：where c > ? and d = ? 那么即使 c 的区分度更高，也必须把 d 放在索引的最前列，即建立组合索引 idx_d_c。

10. 【推荐】防止因字段类型不同造成的隐式转换，导致索引失效。

11. 【参考】创建索引时避免有如下极端误解：

1）索引宁滥勿缺。认为一个查询就需要建一个索引。
2）吝啬索引的创建。认为索引会消耗空间、严重拖慢记录的更新以及行的新增速度。
3）抵制唯一索引。认为唯一索引一律需要在应用层通过“先查后插”方式解决。

(三) SQL 语句

1. 【强制】不要使用 count(列名) 或 count(常量) 来替代 count(*)，count(*) 是 SQL92 定义的标准统计行数的语法，跟数据库无关，跟 NULL 和非 NULL 无关。

说明：count(*) 会统计值为 NULL 的行，而 count(列名) 不会统计此列为 NULL 值的行。

2. 【强制】count(distinct col) 计算该列除 NULL 之外的不重复行数，注意 count(distinct col1 , col2) 如果其中一列全为 NULL，那么即使另一列有不同的值，也返回为 0。

3. 【强制】当某一列的值全是 NULL 时，count(col) 的返回结果为 0；但 sum(col) 的返回结果为 NULL，因此使用 sum() 时需注意 NPE 问题。

正例：可以使用如下方式来避免 sum 的 NPE 问题：SELECT IFNULL(SUM(column) , 0) FROM table;

4. 【强制】使用 ISNULL() 来判断是否为 NULL 值。

说明：NULL 与任何值的直接比较都为 NULL。

1）NULL<>NULL 的返回结果是 NULL，而不是 false。
2）NULL=NULL 的返回结果是 NULL，而不是 true。
3）NULL<>1 的返回结果是 NULL，而不是 true。

反例：在 SQL 语句中，如果在 null 前换行，影响可读性。
select * from table where column1 is null and column3 is not null；而 ISNULL(column) 是一个整体，简洁易懂。从性能数据上分析，ISNULL(column) 执行效率更快一些。

5. 【强制】代码中写分页查询逻辑时，若 count 为 0 应直接返回，避免执行后面的分页语句。

6. 【强制】不得使用外键与级联，一切外键概念必须在应用层解决。

说明：（概念解释）学生表中的 student_id 是主键，那么成绩表中的 student_id 则为外键。如果更新学生表中的 student_id，同时触发成绩表中的 student_id 更新，即为级联更新。外键与级联更新适用于单机低并发，不适合分布式、高并发集群；级联更新是强阻塞，存在数据库更新风暴的风险；外键影响数据库的插入速度。

7. 【强制】禁止使用存储过程，存储过程难以调试和扩展，更没有移植性。

8. 【强制】数据订正（特别是删除或修改记录操作）时，要先 select，避免出现误删除的情况，确认无误才能执行更新语句。

9. 【强制】对于数据库中表记录的查询和变更，只要涉及多个表，都需要在列名前加表的别名（或表名）进行限定。

说明：对多表进行查询记录、更新记录、删除记录时，如果对操作列没有限定表的别名（或表名），并且操作列在多个表中存在时，就会抛异常。
正例：select t1.name from first_table as t1 , second_table as t2 where t1.id = t2.id;
反例：在某业务中，由于多表关联查询语句没有加表的别名（或表名）的限制，正常运行两年后，最近在某个表中增加一个同名字段，在预发布环境做数据库变更后，线上查询语句出现出 1052 异常：
Column 'name' infield list is ambiguous。

10. 【推荐】SQL 语句中表的别名前加 as，并且以 t1、t2、t3、...的顺序依次命名。
    说明：
    1）别名可以是表的简称，或者是依照表在 SQL 语句中出现的顺序，以 t1、t2、t3 的方式命名。
    2）别名前加 as 使别名更容易识别。
    正例：select t1.name from first_table as t1 , second_table as t2 where t1.id = t2.id;

11. 【推荐】in 操作能避免则避免，若实在避免不了，需要仔细评估 in 后边的集合元素数量，控制在 1000 个之内。

12. 【参考】因国际化需要，所有的字符存储与表示，均采用 utf8mb4 字符集，字符计数方法需要注意。
    说明：
    SELECT LENGTH("轻松工作")；--返回为 12
    SELECT CHARACTER_LENGTH("轻松工作")；--返回为 4
    表情需要用 utf8mb4 来进行存储，注意它与 utf8 编码的区别。

13. 【参考】TRUNCATE TABLE 比 DELETE 速度快，且使用的系统和事务日志资源少，但 TRUNCATE 无事务且不触发 trigger，有可能造成事故，故不建议在开发代码中使用此语句。
    说明：TRUNCATE TABLE 在功能上与不带 WHERE 子句的 DELETE 语句相同。

(四) ORM 映射

1. 【强制】在表查询中，一律不要使用 * 作为查询的字段列表，需要哪些字段必须明确写明。
   说明：
   1）增加查询分析器解析成本。
   2）增减字段容易与 resultMap 配置不一致。
   3）无用字段增加网络消耗，尤其是 text 类型的字段。

2. 【强制】POJO 类的布尔属性不能加 is，而数据库字段必须加 is_，要求在 resultMap 中进行字段与属性之间的映射。
   说明：参见定义 POJO 类以及数据库字段定义规定，在 sql.xml 增加映射，是必须的。

3. 【强制】不要用 resultClass 当返回参数，即使所有类属性名与数据库字段一一对应，也需要定义<resultMap>；反过来，每一个表也必然有一个<resultMap>与之对应。
   说明：配置映射关系，使字段与 DO 类解耦，方便维护。

4. 【强制】sql.xml 配置参数使用：#{}，#param# 不要使用 ${} 此种方式容易出现 SQL 注入。

5. 【强制】iBATIS 自带的 queryForList(String statementName，int start，int size) 不推荐使用。
   说明：其实现方式是在数据库取到 statementName 对应的 SQL 语句的所有记录，再通过 subList 取 start，size 的子集合，线上因为这个原因曾经出现过 OOM。
   正例：

```
Map<String, Object> map = new HashMap<>(16);
map.put("start", start);
map.put("size", size);
```

6. 【强制】不允许直接拿 HashMap 与 Hashtable 作为查询结果集的输出。
   反例：某同学为避免写一个<resultMap>xxx</resultMap>，直接使用 Hashtable 来接收数据库返回结果，结果出现日常是把 bigint 转成 Long 值，而线上由于数据库版本不一样，解析成 BigInteger，导致线上问题。
7. 【强制】更新数据表记录时，必须同时更新记录对应的 update_time 字段值为当前时间。
8. 【推荐】不要写一个大而全的数据更新接口。传入为 POJO 类，不管是不是自己的目标更新字段，都进行 update table set c1 = value1 , c2 = value2 , c3 = value3；这是不对的。执行 SQL 时，不要更新无改动的字段，一是易出错；二是效率低；三是增加 binlog 存储。
9. 【参考】@Transactional 事务不要滥用。事务会影响数据库的 QPS，另外使用事务的地方需要考虑各方面的回滚方案，包括缓存回滚、搜索引擎回滚、消息补偿、统计修正等。
10. 【参考】<isEqual>中的 compareValue 是与属性值对比的常量，一般是数字，表示相等时带上此条件；<isNotEmpty>表示不为空且不为 null 时执行；<isNotNull>表示不为 null 值时执行。